

RESUMEN EXTENSO: PREPROCESADORES SASS Y LESS

1. ¿Qué es un preprocesador CSS?

Un **preprocesador CSS** es una herramienta que permite escribir código de estilo usando una sintaxis más avanzada que la de CSS tradicional. Después, ese código se **compila** y se convierte en un archivo **CSS estándar**, que sí puede interpretar el navegador.

Los preprocesadores permiten:

- Usar **variables**
- Crear **bloques reutilizables** (mixins)
- Organizar el código en **múltiples archivos**
- Usar **funciones, operadores, bucles, condiciones**
- Mejorar la **productividad** y el **mantenimiento** del código

Los dos más conocidos son **Sass** y **Less**.

2. Sass

Sass significa **Syntactically Awesome Style Sheets**. Es uno de los preprocesadores más usados y es la base de frameworks como **Bootstrap 4**.

✓ Características principales

- Está escrito en **Ruby**.
- Permite dos sintaxis:
 - **Sass** (sin llaves ni puntos y coma, basada en indentación)
 - **SCSS** (muy similar a CSS, con llaves y puntos y coma)
- Se compila a CSS mediante el comando:

```
sass --watch archivo.scss:archivo.css
```

- Permite variables, mixins, funciones, operadores, bucles, condiciones...

3. Instalación de Sass

Requiere tener Ruby instalado

- **Windows:** descargar Ruby desde rubyinstaller.org
- **Mac:** viene instalado, pero puede actualizarse con `rvm install`
- **Linux:** instalar con `sudo apt-get install ruby`

Instalar Sass

```
sudo gem install sass
```

4. Diferencia entre Sass y SCSS

Sass	SCSS
No usa llaves ni puntos y coma	Usa llaves y puntos y coma
Basado en indentación	Muy similar a CSS
Sintaxis más limpia	Más fácil para quien ya sabe CSS

Ejemplo Sass:

```
$color: #3498AA
body
  color: $color
```

Ejemplo SCSS:

```
$color: #3498AA;
body {
  color: $color;
}
```

5. Cómo funciona Sass con HTML

1. El HTML enlaza un archivo CSS, no SCSS:

```
<link rel="stylesheet" href="estilos.css">
```

- El desarrollador escribe en un archivo **.scss**.
- Sass compila automáticamente el SCSS a CSS usando:

```
sass --watch estilos.scss:estilos.css
```

Cada vez que guardas el SCSS, el CSS se actualiza.

6. Estructura de carpetas en Sass

Es habitual dividir el código en varios archivos SCSS:

- `estilos.scss` (archivo principal)
- `_config.scss`
- `_mixins.scss`
- `_utilities.scss`

El archivo principal importa los demás:

```
@import "config";
@import "mixins";
@import "utilities";
```

7. Variables en Sass

Se declaran con \$:

```
$color-primario: #3498db;  
$fuente: 'Arial';
```

Se usan así:

```
body {  
  color: $color-primario;  
}
```

Ventajas:

- Cambias un valor en un solo sitio
- Mantiene coherencia en el diseño

8. Mixins en Sass

Un **mixin** es un bloque de código reutilizable, parecido a una función.

Crear un mixin:

```
@mixin sizes($w, $h) {  
  width: $w;  
  height: $h;  
}
```

Usarlo:

```
.box {  
  @include sizes(100px, 200px);  
}
```

Permiten:

- Reutilizar código
- Aceptar parámetros
- Personalizar estilos

9. Operadores en Sass

Sass permite usar operadores:

Tipo de operador	Operador	Descripción
Aritmético	+	Suma
	-	Resta
	*	Multiplicación
	math.div()	División
	%	Módulo

Tipo de operador	Operador	Descripción
Comparación	==	Igual
	!=	Distinto
	>	Mayor
		Menor
	>=	Mayor o igual
		Menor o igual
Lógico	&&	AND
	\	OR
	!	NOT

10. Estructuras de control en Sass

@if / @else if / @else

Permiten condiciones:

```
@if $activo {
  color: green;
} @else
```

@for

Itera un número de veces:

```
@for $i from 1 through 5 {
  .caja-#{ $i } {
    width: 20px * $i;
  }
}
```

@each

Itera sobre listas:

```
@each $color in red, blue, green {
  .#{ $color } {
    background: $color;
  }
}
```

@while

Repite mientras se cumpla una condición:

```
$contador: 1;

@while $contador <= 5 {
  .box-#{ $contador } {
    width: 50px * $contador;
  }
  $contador: $contador + 1;
}
```

11. Funciones útiles en Sass

Sass incluye funciones muy potentes, especialmente para colores:

Función	Descripción
<code>darken(color, %)</code>	Oscurece un color
<code>lighten(color, %)</code>	Aclara un color
<code>mix(c1, c2, %)</code>	Mezcla dos colores
<code>complement(color)</code>	Devuelve el color complementario
<code>invert(color)</code>	Invierte el color
<code>percentage(valor)</code>	Convierte un número a porcentaje
<code>grayscale(color)</code>	Convierte a escala de grises
<code>transparentize(color, %)</code>	Aumenta la transparencia
<code>nth(lista, n)</code>	Devuelve el elemento n de una lista

12. Less

Less significa **Leaner Style Sheets**.

Características

- Funciona sobre **JavaScript**
- Se instala con **Node.js**
- Usa variables con **@** en lugar de **\$**
- También permite mixins, funciones y anidación

Instalación

1. Instalar Node.js
2. Ejecutar:

```
npm install -g less
```

Compilación

```
lessc estilos.less estilos.css
```

13. Diferencias principales entre Sass y Less

Less	Sass
Más parecido a CSS	Más avanzado
Funciona con JavaScript	Funciona con Ruby
Compilación manual con lessc	Compilación dinámica con --watch
Variables con @	Variables con \$

14. Conclusión general

Sass y Less son herramientas que **mejoran enormemente el desarrollo CSS**, permitiendo:

- Código más limpio
- Reutilización
- Automatización
- Mantenimiento más fácil
- Estilos más dinámicos y potentes

Sass es el más completo y extendido, mientras que Less es más simple y cercano a CSS.